

Day 71 Research Log -- Ceiling Broken, Scale Re-Read (June 14, 2026)

The day that opened with Pack 247d taxonomy cache fix (build `_isa_inverse` directly from SEED dict, bypass substrate-query pollution -- top-1 +0.002 / top-5 +0.041 / UNSEEN 4x relative, confirming clean signal works where coverage exists); caught and fixed a permanent hardcoded bug in `integrate.py` (constructor took `d=400` then ignored it, instantiating `FlatMemory/MultiRoleMemory/FrameField` at `d=512` -- `load_ikg` silently corrected MR via `.ikg`-saved dim but left `FrameField` at `d=512`, causing dimension mismatch on first frame routing call); shipped `frame_field` init + post-hoc `frame_vocab` filter (Pack 247e -- frame routing works at 8/8 frames after warm with entropy 2.92/3.0 near maximum); wired crystal triple prior + holographic context blend permanent (Pack 247h, all 4 wires individually positive at ~+0.004 top-1 each, no destructive interference under Fix C protection); audited every module in `ikigai/cognition/` for predict-time value (Pack 247j -- 91 modules classified into WIRED-LIVE / WIRED-NULLED-by-flat_only / lazy on-demand / module-level-helpers / genuinely-dead, the 5 "None" modules confirmed intentionally nulled by `flat_only=True` branch, no critical missing wire); built `HSPColumnPooler` from scratch in `ikigai/cognition/hsp.py` (Hyperdimensional Spatial Pooling, $M=256/k=20$, 100% coverage); Phase 1 failed catastrophically on lex-trigram input (peer Jaccard 0.034 inverted below random 0.061 because `ck.key(word)` is a hash, not a semantic vector); Phase 1c added common-mode removal + HTM-style column boost and hit 6.10x peer/random topology ratio; Phase 2 wired HSP into predict (marginal +0.006 top-1, slightly above the +0.004 correlated ceiling but not the unblock we wanted); paste-back research return (Day 71, attached `hsp.py`) diagnosed the wire as boosting *semantic neighbors* of current when right next-word is *syntactic successor* of current's class -- the topology was right but the wire direction was wrong; Phase 3 syntactic HSP swapped input to NEXT-only role-recall with HSP-side common-mode and jumped topology to 20.42x peer/random (no semantic leak); Phase 4-A added HTM Temporal Memory col-pair successor matrix $P[i,j]$ learned from train bigrams, sanity check revealed leakage to corpus-frequent targets (april/august dominate every row -- raw row-normalization insufficient); second research return (paste-back, no files supported -- inlined all numbers) named PPMI normalization with $\alpha=0.75$ smoothing for transition matrix + read-time common-mode subtraction at substrate read time + per-class FHRR transition operator + polynomial/softmax sharpening + POS oracle diagnostic for info-theoretic vs extractability; Phase 4-B shipped both PPMI + read-time CM and **BROKE THE CEILING** -- top-1 0.051 $\rightarrow$ 0.065 (+0.014, 3.5x the previous +0.004 wire ceiling), top-5 0.072 $\rightarrow$ 0.126 (+75%), SEEN bigrams 0.323 $\rightarrow$ 0.398 (+23% relative), UNSEEN 0.0010 $\rightarrow$ 0.0038 (3.8x relative but still small absolute, real signal); **BAKE PERMANENT** shipped `build_predict_common_mode()` + `build_hsp()` + `fit_hsp_transitions()` + `attach_hsp()` methods on `IkigaiOrganism`, `_predict_v_common` + `hsp` added to `_PERSIST_ATTRS`, `predict_with_wires` auto-uses cached state on any future load, `organism.ikg` now 58.02 MB carrying full Phase 4-B stack; Phase 4-C POS oracle (NLTK `pos_tag` over vocab, 11 coarse POS classes from PTB, expected-next-POS class transition matrix from train bigrams) returned UNSEEN = $0/2091 = 0.0000$ (oracle ZERO unseen recovery, top-1 actually dropped 0.051 $\rightarrow$ 0.047), proving the 2K-

scale UNSEEN floor is INFO-THEORETIC -- substrate genuinely doesn't have signal for pairs absent from absorb distribution, even with perfect POS labels; then 10K SCALE TEST inverted everything: at 5000 articles / 25K sentences / 116852 train bigrams / 29244 heldout predictions, RAW substrate alone delivered top-1 0.071 and UNSEEN 0.0553 (55x the 2K UNSEEN floor!), while FULL STACK with all wires gave top-1 0.063 and UNSEEN 0.0361 -- the wires actively HURT at realistic scale because they reorder a substrate-correct top-1 toward a wire-preferred wrong candidate; substrate-confidence gate added permanent to `predict_with_wires` (`wire_confidence_gate` param: skip all wires when substrate top-1 margin over top-2 exceeds threshold) to preserve the substrate-strong cases at scale while keeping wires for the uncertain ones. Day 71 ended with the project's mental model rewritten: the substrate is much stronger than the narrow-eval ceiling-break framing suggested; the wires deliver real lift only when substrate is weak; THE FLIP is now even more compelling because absorbing LLM topology onto a substrate that already hits 7.1% top-1 at scale will compound multiplicatively, not just rescue a weak baseline.

TL;DR

Day 71 (June 14) shipped the locked Day 70 sequence (247d → 247e → 247f → 247g) faster than expected and uncovered a structural re-read of the entire project's empirical regime. The taxonomy cache fix (247d) confirmed clean ISA wire works where vocab coverage exists. The hardcode bug fix in `integrate.py` (constructor `d=400` was being ignored by every internal `FlatMemory(d=512)` / `FrameField(d=512)` etc. -- now `d=d` throughout) cleaned a silent corruption path that would have bitten every multi-d experiment. Frame field initialization + isolated routing test (247e) verified 8/8 frame coverage with near-maximum entropy. Crystal triple prior + holographic context blend (247h) wired both at predict, every wire individually positive at ~+0.004 top-1, no destructive interference. Full /cognition audit (247j) classified 91 modules — no critical missing wires; the 5 None modules (belief, vsa, tom, cf, cwm) are nulled by `flat_only=True` design choice.

Pack 247g HSP build went through three phases. Phase 1 failed because lex-trigram input gives the wrong topology (hash distance, not semantic). Phase 1c added common-mode removal + HTM column boost and hit 6.10x peer/random Jaccard. Phase 2 wired HSP into predict at marginal +0.006 top-1. Paste-back research diagnosed semantic-neighbor wire as wrong direction; right next-word is syntactic successor not semantic peer. Phase 3 swapped input to NEXT-only recall and topology jumped to **20.42x** peer/random ratio. Phase 4-A added HTM temporal-memory col-pair successor matrix $P[i,j]$; sanity check revealed leakage to corpus-frequent targets (april/august dominated every row).

Second research return (paste-back inline) prescribed **PPMI normalization** with $\alpha=0.75$ smoothing + **read-time common-mode subtraction** at substrate read time, with full math given for both. Phase 4-B implemented both and **broke the ceiling**: held-out top-1 0.051 → 0.065 (+0.014, 3.5x the +0.004 wire ceiling). Top-5 0.072 → 0.126. SEEN 0.323 → 0.398. UNSEEN 0.0010 → 0.0038 (3.8x). The bake permanent shipped `build_predict_common_mode` / `build_hsp` / `fit_hsp_transitions` / `attach_hsp` on `IkigaiOrganism`, with `_predict_v_common` and `hsp` added to `_PERSIST_ATTRS`. `organism.ikg` now 58.02 MB carrying the full Phase 4-B

stack; any future `IkigaiOrganism.load_ikg()` → `predict_with_wires()` automatically uses cached CM + HSP without explicit kwargs.

Phase 4-C POS oracle diagnostic (NLTK pos_tag, 11 coarse classes) returned **zero unseen recovery**: oracle UNSEEN = 0/2091, top-1 dropped 0.051 → 0.047. This proves the 2K-scale UNSEEN floor is **information-theoretic** -- substrate genuinely doesn't have signal for absent bigrams, even with perfect POS labels. Per the research's diagnostic spec: Outcome B (UNSEEN ~ 0.001 with oracle) means d=400 might be too narrow OR substrate absorb distribution didn't cover the unseen pairs.

Then 10K scale test inverted the framing. At 5000 articles / 116852 train bigrams / 29244 heldout predictions: **RAW substrate alone hits 0.071 top-1, UNSEEN 0.0553 (55x the 2K floor)**. FULL STACK with all wires: top-1 0.063, UNSEEN 0.0361. The wires HURT at realistic scale. The 0.057 ceiling and 0.001 UNSEEN floor measured at narrow 2K eval were artifacts of narrow eval, not real substrate limits. The wires were tuned for the artifact regime and now reorder correct substrate top-1 to wire-preferred wrong candidates.

Substrate-confidence gate added permanent: `wire_confidence_gate` param skips all wires when substrate top-1 margin over top-2 exceeds threshold. Preserves substrate-strong cases at scale, keeps wires for uncertain cases. Day 71 ended with the mental model rewritten: substrate is much stronger than narrow eval suggested; wires need confidence gating to add value at scale; THE FLIP (Pack 248) compounds onto an already-strong substrate, not a weak one.

Packs shipped Day 71

Pack	Build	Key result
247d	Taxonomy cache fix (build <code>_isa_inverse</code> from SEED dict, skip substrate query)	top-1 +0.002, top-5 +0.041, UNSEEN 4x relative. Clean signal verified. 6.6% vocab coverage limits absolute lift.
hardcode fix	<code>integrate.py</code> <code>__init__</code> was ignoring <code>d</code> param. Set <code>d=d</code> everywhere instead of <code>d=400/512</code> . Permanent.	Fixes silent dimension mismatch on <code>FrameField</code> (was <code>d=512</code> vs MR <code>d=400</code>) and 8 other modules
247e	<code>FrameField</code> init + isolated routing + <code>frame_vocab</code> post-hoc filter wire	8/8 frames active, routing entropy 2.92/3.0 ≈ max, <code>frame_vocab</code> 7551 tokens. top-1 +0.004, top-5 +0.040, UNSEEN 3.3x
247f	<code>set_taxonomy_seed()</code> + <code>predict_with_wires()</code> permanent on <code>IkigaiOrganism</code> . Fix C (<code>protect_taxonomy</code> + <code>frame_score_min</code> gate).	Roundtrip verified bit-perfect. Combo no longer destructive.
247h	Crystal triple prior + holographic context blend	Crystal 42% coverage (>> taxonomy 6.6%). Crystal

Pack	Build	Key result
	permanent	+0.005 top-1. Holo +0.004. All 4 wires correlated, ~0.115 top-5 each.
247i/j	/cognition audit (91 modules classified)	24 WIRED-LIVE, 14 NULLED-by-flat_only, 7 lazy-on-demand, 5 module-helpers, ~25 orphan/dead, 10 utility. No critical gap.
247g Phase 1	HSPColumnPooler scaffold (M=256, k=20, HTM-style, complex-phasor adapted) on lex-trigram input	FAIL. Peer Jaccard 0.034 < random 0.061. Lex hash is wrong topology.
247g Phase 1c	Add common-mode removal + HTM column boost. Switch to behavioral input vector	PASS. Peer 0.204, random 0.034, ratio 6.10x . 100% coverage. Topology emerging.
247g Phase 2	Wire HSP into predict_with_wires (do_hsp param, hsp_mode='neighbor')	top-1 +0.006, marginal above pre-HSP +0.004. Ceiling not broken.
research return 1 (paste-back, day 71 attached hsp.py)	Diagnosis: semantic-neighbor wire wrong direction; need syntactic-successor	Recommendation: NEXT-only input, M=512/k=10, boost=2.5, FHRR transition operator, Resonator factorization
247g Phase 3	Switch input to recall(NEXT) only with HSP common-mode. M=512, k=10, boost=2.5.	Topology 20.42x peer/random. But predict barely budged (0.053). Wire direction still wrong: boosts neighbors of current, not successors.
247g Phase 4-A	Add fit_transitions() building HTM col-pair P[i,j] + expected_next_sdr(). Add hsp_mode='successor' to predict_with_wires.	Flat. Sanity check revealed leakage: through → august/april, large → august/april, would → august/april. Wiki dates dominate every row.
research return 2 (paste-back inline, no files supported)	Diagnosis: substrate READ corruption by common-mode background. Math given: PPMI($\alpha=0.75$), read-time CM subtraction (linearity proof), polynomial + softmax sharpening, per-class FHRR R operator, POS oracle diagnostic protocol	Two cheap orthogonal fixes named: PPMI on P + read CM. Both ≤ 30 min build.
247g Phase 4-B	PPMI normalization on transition matrix ($\alpha=0.75$) + read-time common-mode	CEILING BROKEN. top-1 0.051 → 0.065 (+0.014, 3.5x). top-5 +0.054. SEEN +0.075.

Pack	Build	Key result
	subtraction (<code>r_clean = r_next - v_common</code>) in <code>predict_with_wires</code>	UNSEEN 3.8x. PPMI sanity: april/august rank 5 not 1.
247g BAKE PERMANENT	<code>build_predict_common_mode()</code> + <code>build_hsp()</code> + <code>fit_hsp_transitions()</code> methods + <code>_predict_v_common</code> + <code>hsp</code> in <code>_PERSIST_ATTRS</code> . Auto-use cached state in <code>predict_with_wires</code> .	Roundtrip clean: fresh load auto-fires Phase 4-B without kwargs. <code>organism.ikg = 58.02 MB</code> .
247g Phase 4-C	NLTK <code>pos_tag</code> whole vocab (11 coarse PTB classes). Build POS transition matrix. Oracle wire: predict next-POS, restrict candidates to expected POS class.	UNSEEN = 0/2091 with oracle. Top-1 dropped 0.051 → 0.047. Confirms 2K UNSEEN floor is INFO-THEORETIC: substrate doesn't have unseen-pair signal even with perfect POS.
247g SCALE 10K	5000 articles / 25K sentences / 116852 train bigrams / 29244 heldout. Rebuild PPMI matrix on this distribution.	Substrate at scale is FAR stronger than narrow eval showed. RAW top-1 0.071 , UNSEEN 0.0553 (55x the 2K floor). FULL STACK: 0.063 / 0.0361. Wires HURT at scale.
247g Phase 4-D	<code>wire_confidence_gate</code> param added permanent: skip all wires when substrate top-1 margin over top-2 exceeds threshold	Preserves substrate-strong cases. Wires gate to uncertain cases.

Architectural notes (5 lessons)

1. **Eval regime matters more than wires.** The 0.057 → 0.065 ceiling-break at 2K corpus was real for that regime but inverted at 10K. At realistic scale the substrate alone hits 0.071 top-1 / 0.0553 UNSEEN. Every wire experiment from Pack 245C onward was tuned against an artifact regime. **Future packs must validate at 10K BEFORE shipping permanent.**
2. **Common-mode removal is the universal sharpener.** Worked at HSP input (Phase 1c → 6.10x), worked at HSP input with NEXT-only (Phase 3 → 20.42x), worked at substrate READ (Phase 4-B → +0.011 top-1 alone). Linearity preserves role-binding algebra. Single most impactful primitive of Day 70-71. Should be the **FIRST** thing tried on any future signal-extraction problem.
3. **PPMI > raw counts > row-normalization.** Successor leakage in HTM col-pair matrices is fundamental. Frequency-biased smoothing $\alpha=0.75$ from word-embedding literature carries over cleanly. Row-norm alone scales only outbound; doesn't fix inbound popularity bias.

4. **Substrate-confidence gating is the real wire architecture.** Wires that boost are dangerous: they can move a correct substrate top-1 to a wire-preferred wrong candidate. Confidence gate (skip wires when substrate top-1 - top-2 margin > threshold) is the structurally correct pattern. Future wires should default to gated.
 5. **POS oracle as info-theoretic floor test is a free diagnostic.** NLTK pos_tag, 11 classes, 30s build. If oracle UNSEEN $\approx 0.001 \rightarrow$ info-theoretic; if $> 0.01 \rightarrow$ extractability. Should run at every scale step.
-

Status at Day 71 close

organism.ikg state (58.02 MB):

- substrate: multi-bank 384 MiB FIXED, b_lang energy 445.7M, 2878 vocab, d=400
- _predict_v_common cached (Phase 4-B)
- hsp attached (M=512, k=10, PPMI-normalized P, common-mode, 100% coverage)
- _taxonomy_seed + _taxonomy_word_to_class (10 classes, 191 vocab covered)
- FrameField warm (8/8 frames, top tokens locked at 64)
- 1591 crystal triples
- 9 vs_fsm state items persist (schemas, _isa_inverse etc.)
- 4 bridge classes restored on load (_wm_sys, _cg, _ec, _cas)

integrate.py permanent additions (Pack 247d-h, 247g):

- set_taxonomy_seed(seed_dict) — install hand-curated taxonomy + clean _isa_inverse
- predict_with_wires(prev, current, ...) — 1 entry point, 5 wires, confidence gate
- build_predict_common_mode() — compute + cache v_common
- build_hsp() — fit syntactic HSP (Phase 3 doctrine: NEXT-only, common-mode, M=512/k=10)
- fit_hsp_transitions() — PPMI-normalized successor matrix
- attach_hsp() — attach pre-built HSP pooler
- _PERSIST_ATTRS extended with _taxonomy_seed, _taxonomy_word_to_class, _predict_v_common, hsp
- hardcoded d=400/512 bug fixed throughout __init__

ikigai/cognition/ permanent additions:

- hsp.py (new): HSPColumnPooler class. Common-mode removal, HTM column boost, fit_vocab + fit_transitions (PPMI + α -smoothing), expected_next_sdr, jaccard, nearest. ~250

lines.

Cat-1 numbers (next-token prediction):

Scale	Config	top-1	top-5	UNSEEN
2K narrow	RAW substrate	0.051	0.072	0.0010
2K narrow	Phase 4-B FULL STACK	0.065	0.126	0.0038
10K realistic	RAW substrate	0.071	0.082	0.0553
10K realistic	FULL STACK (ungated)	0.063	0.109	0.0361
10K realistic	(gated -- to test Day 72)	TBD	TBD	TBD

Doctrine:

- Common-mode removal is universal. Apply at any signal-extraction surface.
 - PPMI > row-norm for any co-occurrence matrix.
 - Wires need confidence gating at scale.
 - Substrate at realistic scale (10K) is FAR stronger than narrow eval (2K) showed. THE FLIP compounds onto strength, not weakness.
 - ORGANISM, NOT SUBSTRATE (Rule Zero held).
 - Path D rejected (Day 70 close), Path E (THE FLIP) locked.
-

What did not ship

- **Phase 4-D gated test at 10K** — confidence gate added permanent but per-wire breakdown experiment queued, not run.
 - **Phase 4-E softmax sharpening test at 10K** — script ready, awaiting scale-10K queue.
 - **FHRR per-class R transition operator** — research recommendation. Bigger build (1-2 days). Deferred to Day 72+ if confidence gate doesn't recover scale wins.
 - **VS-BIND boundary-relative tags** — research mentioned. Deferred to 5 Heavy Inventions queue.
 - **POS oracle at 10K** — only ran at 2K. Should run at 10K to confirm info-theoretic vs extractability at scale.
 - **Frame field persistence wire** — frame_field state persists in organism.ikg via existing save_ikg, but auto-warm-on-load wasn't shipped.
-

Open questions (5)

1. **Which specific wire hurts at 10K?** Per-wire breakdown queued (script ready: `day71_pack247g_phase4d_per_wire_10k.py`). Hypothesis: HSP successor + frame filter most aggressive at reordering, most likely culprits.
 2. **Does confidence gate recover the scale loss?** Phase 4-D added `wire_confidence_gate` param. Should preserve substrate-strong cases and only apply wires when uncertain. Test pending.
 3. **What's the info-theoretic floor at 10K?** POS oracle returned 0/2091 at 2K. Need to re-run at 10K to see if oracle UNSEEN > 0.01 at scale.
 4. **Should we switch eval default to 10K?** Every wire-tuning experiment after Day 71 should validate at 10K minimum. 2K artifacts misled 5+ packs.
 5. **Pack 248 timing?** THE FLIP becomes more attractive given substrate at scale is at 0.071 already. Absorbing LLM topology = multiplicative win, not rescue. Could start scaffolding T2S Llama-3 absorb path while gating tests finish.
-

What we learned (3 major lessons)

1. **Narrow eval artifacts mislead architecture.** The 0.057 ceiling diagnosed by Day 70 research and the +0.004 correlated wire ceiling we hit at 2K were both real at 2K and irrelevant at 10K. Every "ceiling break" framing must be validated at realistic scale BEFORE doctrine-locking.
 2. **The substrate is much stronger than we thought.** At 10K with the SAME organism.ikg, RAW substrate hits 0.071 top-1 / 0.0553 UNSEEN. The substrate has been generalizing all along — we were measuring it on a narrow eval that obscured the signal. This rewrites the whole "we need to build wires to compensate" narrative.
 3. **Research workflow validated.** Two paste-back research returns Day 71 each delivered NAMED primitives + parameter ranges + math. PPMI + read CM came from research-2026-06-14 paste-back. Without that, we would have built more wires guessing. The new `feedback_research_workflow.md` memory rule formalizes this for future packs.
-

Appendix A -- Inventions / Primitives shipped Day 71

#	Name	Where	What
53	HSPColumnPooler	ikigai/ cognition/ hsp.py	Hyperdimensional Spatial Pooler with common-mode removal + HTM column boost + PPMI fit_transitions. Fixed footprint, online, backprop-free.

#	Name	Where	What
54	Read-time common-mode subtraction	<code>integrate.py:predict_with_wires</code>	<code>r_clean = r_next - v_common</code> . Universal substrate-read sharpener. Linearity preserves role-binding.
55	PPMI normalization with α -smoothing	<code>hsp.py:fit_transitions</code>	<code>PPMI[i, j] = max(0, log2(P[i, j] * N_α / (C_src[i] * C_tgt[j]^α)))</code> with $\alpha=0.75$. Fixes successor leakage to corpus-frequent targets.
56	Fix C wire interaction protection	<code>integrate.py:predict_with_wires</code>	Boosted indices protected from frame penalty. Frame-routing-confidence gate. Prevents combo destructive interference.
57	Polynomial sharpening	<code>predict_with_wires</code>	$\tilde{c} = \text{sign}(c) * c ^\beta$. Phase 4-E preview. Saturated above CM (no marginal lift at $\beta=1.5/2.0/3.0$).
58	Softmax sharpening	<code>predict_with_wires</code>	$\tilde{c} = \text{softmax}(\lambda * c)$, $\lambda \geq 10$. Research-recommended. Awaiting eval.
59	Substrate-confidence wire gate	<code>predict_with_wires</code>	<code>wire_confidence_gate</code> : skip all wires when substrate top-1 margin over top-2 > threshold. Preserves strong substrate cases at scale.
60	Behavioral key adapter pattern	(experiments)	<code>NextOnlyKey.key(w) = mr.recall(w, NEXT) / mr.recall .mean()</code> . Right input vector for syntactic HSP.
61	Permanent organ wires + bake	<code>integrate.py</code>	<code>set_taxonomy_seed, build_hsp, build_predict_c</code>

#	Name	Where	What
			ommon_mode, attach_hsp, fit_hsp_transitions — full Phase 4-B baked into organism.ikg.
62	NLTK POS oracle diagnostic protocol	experiments + research doctrine	Train HSP on perfect ground-truth POS labels; measure UNSEEN. Outcome A (>0.01) = extractability, B (~0.001) = info-theoretic. 30-min, free signal.

Appendix B -- The hardcoded bug (formal documentation)

integrate.py: __init__ signature was `def __init__(self, d=400, flat_only=False)`. The `self.d = d` captured the param but every internal instantiation ignored it:

```
# Before fix:
self.flat      = None if flat_only else FlatMemory(d=512, M=16384, k=64,
seed=114)
self.unified    = MultiRoleMemory(d=512, M=16384, k=64, seed=114)
self.frames     = FrameField(d=512, K=8, top_n=64, seed=42, alpha=0.5)
self.curiosity  = CuriosityDrive(d=400)
self.tom        = TheoryOfMindSandbox(d=400)
self.vsa        = VSACalculus(d=400)
self.belief     = BeliefField(d=400)
self.verifier   = SelfVerifier(d=400, threshold=0.5)
self.proof_gen  = ProofCarryingGenerator(d=400)
```

The organism.ikg on disk was saved at `d=400` (Pack 245G+ retrains). On `load_ikg`, the .ikg-saved `MultiRoleMemory` state restored MR at `d=400`, OVERWRITING the constructor's `d=512` instance. But `FrameField` was NOT restored from .ikg in the same path (frames pickled separately, came back at whatever the constructor created -- `d=512`). Result: `org.frames.d == 512` but `org.unified.d == 400`. First `route_passage(tokens, mr.ck)` call: `accum = np.zeros(512)`; `accum + ck.key(t)` where `ck.key` returns `d=400` → `ValueError shape (512,) + (400,)`.

Fix (permanent):

```
self.flat      = None if flat_only else FlatMemory(d=d, M=16384, k=64,
seed=114)
self.unified    = MultiRoleMemory(d=d, M=16384, k=64, seed=114)
self.frames     = FrameField(d=d, K=8, top_n=64, seed=42, alpha=0.5)
self.curiosity  = CuriosityDrive(d=d)
self.tom        = TheoryOfMindSandbox(d=d)
self.vsa        = VSACalculus(d=d)
```

```
self.belief      = BeliefField(d=d)
self.verifier    = SelfVerifier(d=d, threshold=0.5)
self.proof_gen   = ProofCarryingGenerator(d=d)
```

This was a silent corruption path that would have bitten every multi-d experiment. Prince caught it directly ("why hardcode?") inside Pack 247e. Now fixed in one commit, all 9 hardcoded d= values normalized to d=d.

Appendix C -- 10K scale shock (formal record)

The 10K scale test (day71_pack247g_scale_10k.py) was meant to validate Phase 4-B's ceiling break. It validated something else.

Configuration: 5000 simple_wiki articles → 25K sentences → 20K TRAIN / 5K HELDOUT → 116852 train bigrams → 29244 heldout predictions over 2739 candidates. organism.ikg loaded fresh (Phase 4-B baked stack). PPMI transition matrix rebuilt on TRAIN distribution.

Result:

RAW substrate	top-1 0.071	top-5 0.082	SEEN 0.097	UNSEEN 0.0553
FULL STACK	top-1 0.063	top-5 0.109	SEEN 0.110	UNSEEN 0.0361

Reading:

- RAW substrate UNSEEN 0.0553 at 10K vs 0.0010 at 2K = 55x relative increase. Same substrate. The substrate has been generalizing all along — narrow 2K eval just couldn't see it.
- FULL STACK at 10K is WORSE than RAW: top-1 -0.008, UNSEEN -0.0192 (35% relative drop).
- The wires were tuned at 2K where substrate is weak. At 10K substrate is strong enough that wire reordering does net damage.
- top-5 still favors FULL STACK (0.109 vs 0.082) suggesting wires help DIVERSITY of top-5 but hurt CORRECTNESS of top-1.

Implication: every wire-tuning pack from 245C onward was optimizing against the wrong regime. The 5.4% structural floor diagnosed by Day 70 research may have been a narrow-eval artifact, not a fundamental flat-VSA property. THE FLIP becomes even more attractive — absorbing LLM topology onto a 0.071-baseline substrate is multiplicative, not rescue.

Action: substrate-confidence gate added to `predict_with_wires`. Per-wire breakdown queued. Eval default moves to 10K. Future doctrine: validate at 10K BEFORE locking any wire.